

AGRICONNECT FARM-TO-MARKET PLATFORM WITH MOBILE MONEY WALLET

PHASE 6: NORMALIZATION AUDIT

Assignment 3 Material

Purpose: Identify anomalies in the proposed single-table design and demonstrate how the Phase 3 AgriConnect schema supports normalization through Third Normal Form (3NF).

1. Introduction

Database normalization organizes data into related tables to reduce unnecessary duplication and protect consistency. This audit examines the lecturer's proposed *bad_orders* table, explains its update, insert and delete anomalies, and uses the Phase 3 schema to demonstrate 1NF, 2NF and 3NF. It also discusses controlled denormalization for future reporting.

2. Proposed Bad Design

```
CREATE TABLE bad_orders (  
  order_id INT, buyer_name VARCHAR(100), buyer_phone VARCHAR(13),  
  farmer_name VARCHAR(100), product_name VARCHAR(100), category VARCHAR(20),  
  unit_price DECIMAL(10,2), qty DECIMAL(10,2), town VARCHAR(60)  
);
```

This table combines order, buyer, farmer, product and town details. If an order contains multiple products, some buyer, farmer or town information may be repeated. This repetition creates anomalies.

3. Update Anomaly

If a farmer's phone number changes, every row containing that farmer may need updating. If some rows are missed, the database can show inconsistent phone numbers for the same farmer. In Phase 3, user details are stored in *users* and products refer to the farmer using *products.farmer_id*, so the farmer's phone is maintained in one user record.

4. Insert Anomaly

In the bad design, a new product may not be recorded until a buyer orders it, because the table is built around orders. In Phase 3, a product can be inserted into *products* before it appears in any *order_items* record. This allows products to exist in the catalogue before their first sale.

5. Delete Anomaly

If the last order row containing a particular farmer or product is deleted, the only stored copy of those details may disappear. Phase 3 separates users, products, orders and order items. Removing an order line therefore does not inherently remove the farmer's user record or the product record; foreign-key constraints help control related deletions.

6. Functional Dependencies in Phase 3

A functional dependency $X \rightarrow Y$ means a value of X determines one value of Y . The following examples show key dependencies in the schema.

Table	Functional dependency	Meaning
users	$user_id \rightarrow full_name, phone, email, role, town_id, created_at$	A user ID identifies one user's details.
towns	$town_id \rightarrow town_name, district$	A town ID identifies its town information.
products	$product_id \rightarrow farmer_id, product_name, category, unit, unit_price, stock_qty, is_active$	A product ID identifies the product attributes.
orders	$order_id \rightarrow buyer_id, order_date, status, delivery_address, total_amount$	An order ID identifies one order's attributes.
order_items	$order_item_id \rightarrow order_id, product_id, qty, unit_price, line_total$	An order-item ID identifies one order line.

7. First Normal Form (1NF)

1NF requires each field to contain one atomic value rather than a list or repeating group. AgriConnect stores each purchased product as a separate row in *order_items*, with its own product, quantity and purchase-time unit price. It does

not store a list of products inside one order field.

8. Second Normal Form (2NF)

2NF requires a table to be in 1NF and every non-key attribute to depend on the whole candidate key, not only part of a composite key. Most Phase 3 tables use single-column primary keys, so partial dependency on part of a composite primary key does not arise in those tables. Product descriptions such as name, category and farmer ID are stored in *products*, rather than copied into every *order_items* row. Order lines keep order-specific quantity and the unit price frozen at purchase time.

9. Third Normal Form (3NF)

3NF requires a table to be in 2NF and to avoid non-key attributes depending transitively on a key through another non-key attribute. For example, *users* stores *town_id*, while town name and district are stored in *towns*. Similarly, a product stores *farmer_id* rather than repeating the farmer's name and phone number. This separates facts about towns and users from facts about products.

10. Absence of Partial and Transitive Dependencies

The schema demonstrates the absence of partial dependencies mainly through single-column primary keys and by placing attributes in the tables representing the entities they describe. For any table with a composite candidate key, the same rule must be checked against the full key; single-column primary keys alone do not prove every possible dependency. The schema also reduces common transitive dependencies by separating towns, users, products, orders and order items. Town names and districts depend on *town_id* in *towns*; farmer details belong to a user record and products reference that record through *farmer_id*.

11. Justified Denormalization for Reporting

In a later analytics phase, AgriConnect may create a reporting table such as *fact_order_lines* that duplicates selected values such as product name, category or farmer name alongside order-line measures. This can reduce joins and improve read performance for dashboards. The trade-off is extra storage and a risk of inconsistent duplicated values if source details change or the reporting table is not refreshed correctly. Denormalization should therefore be controlled and justified for read-heavy analytics, while the normalized Phase 3 tables remain the transactional source of truth.

12. Conclusion

The proposed *bad_orders* table mixes facts about several entities and creates update, insert and delete anomalies. The Phase 3 schema addresses these problems by separating users, towns, products, orders and order items and linking them through keys. Its structure demonstrates the aims of 1NF, 2NF and 3NF: atomic values, no partial dependencies on part of a composite key, and reduced transitive dependencies. Controlled denormalization may later support faster reporting if consistency and maintenance costs are managed.